

UNIVERSIDAD SANTO TOMÁS

Sistema colaborativo de robots móviles para asistencia de orientación en entornos interiores con múltiples pisos

Realizado por:

Santiago Hernández Ávila
Jonny Alejandro Mejía León



Informe de Avance - Semana 13

Fase 3: Cierre del entorno virtual — Fase 4: Inicio del desarrollo progresivo

Dirigido por:

Ing. Armando Mateus Rojas, Msc.
Ing. Nestor Ivan Ospina, Msc.
Ing. Oscar Mauricio Gélvez Lizarazo, Msc.

GED (Grupo de Estudio y Desarrollo en Robótica)
Facultad de Ingeniería Electrónica
División de Ingenierías
Bogotá, mayo de 2026

1. Introducción

El presente informe documenta el avance correspondiente a la **Semana 13** del cronograma de actividades. Esta semana marca el inicio formal de la **Fase 4 (Desarrollo progresivo de la solución)** conforme al cronograma del anteproyecto, y se sustenta sobre el cierre de Fase 3 logrado en la Semana 12 con la validación experimental de la plataforma DeepRacer en ROS 2 Humble.

El trabajo de esta semana se orientó a tres líneas complementarias: la consolidación del gemelo digital del entorno de pruebas mediante el modelado de los pasillos de la Universidad Santo Tomás en Gazebo, la implementación de un primer comportamiento reactivo (*wall following*) sugerido por la dirección del proyecto a partir de algoritmos existentes en la literatura, y la investigación de alternativas de software de comunidad para la futura fase de implementación en hardware real.

2. Objetivos de la Semana

- Modelar el entorno virtual del pasillo de la Universidad Santo Tomás en Gazebo Classic 11 utilizando el Building Editor.
- Validar la integración del DeepRacer dentro del entorno modelado mediante pruebas de spawn y control manual (teleoperación).
- Implementar un primer nodo de control reactivo (*wall follower*) aplicando algoritmos existentes en la literatura, como ejercicio de integración entre el LiDAR y la cinemática Ackermann del DeepRacer.
- Evaluar el repositorio `deepracer-custom-car` sugerido por la codirección como alternativa para la futura fase de implementación física sobre Raspberry Pi 4.

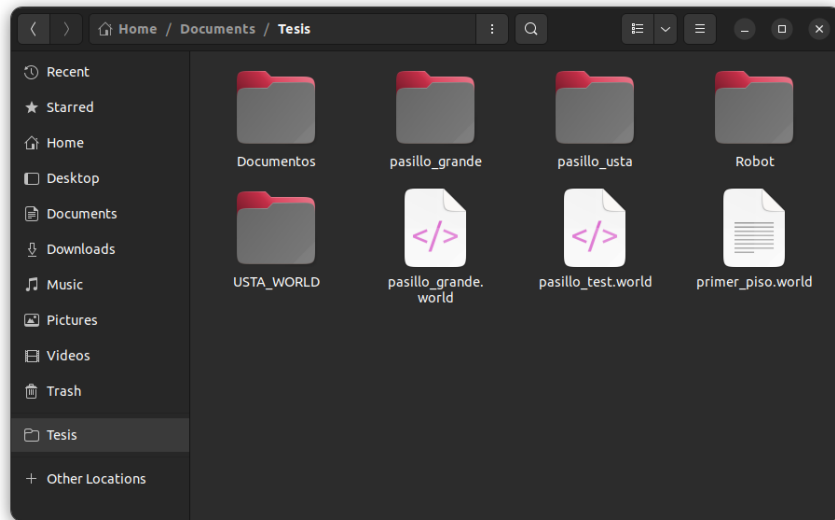
3. Modelado del entorno virtual

3.1. Creación del modelo en Gazebo Building Editor

Se modeló un primer pasillo representativo de los corredores de la Universidad Santo Tomás utilizando el Building Editor de Gazebo Classic 11. El modelo, denominado `pasillo_usta`, corresponde a un rectángulo de aproximadamente 7 m de largo por 3 m de ancho, dimensiones representativas de los corredores interiores de la edificación objetivo.

3.2. Estructura de archivos y configuración del entorno

El modelo se guardó en una estructura de archivos limpia y compatible con los estándares de Gazebo, como se muestra en la Figura 1. La carpeta del modelo contiene los dos archivos requeridos por el simulador: `model.config` con los metadatos del modelo, y `model.sdf` con la descripción geométrica de las paredes.



[Diagrama de la estructura de archivos: carpeta Tesis con subcarpetas pasillo_usta y pasillo_grande (cada una con model.config y model.sdf), junto con los archivos pasillo_test.world y pasillo_grande.world]

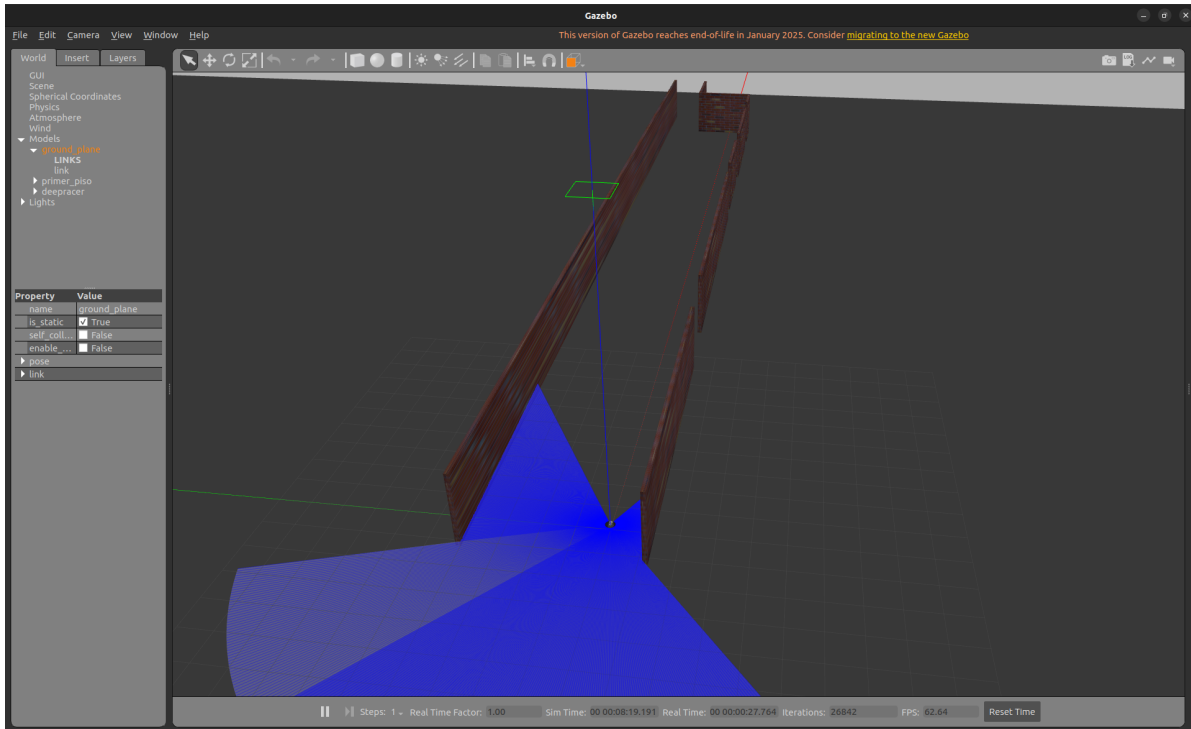
Figura 1: Organización de archivos del entorno virtual en el directorio `/home/santiago/Documents/Tesis/`.

Adicionalmente se configuró la variable de entorno `GAZEBO_MODEL_PATH` apuntando al di-

rectorio padre, lo cual permite que el simulador detecte el modelo de forma automática al ser referenciado desde un archivo de mundo. Se generó el archivo `pasillo_test.world` que incluye el modelo del pasillo junto con el plano de suelo y la fuente de iluminación estándar.

3.3. Validación del entorno con el DeepRacer

Con el entorno modelado se procedió a la validación funcional mediante el lanzamiento del DeepRacer migrado a ROS 2 Humble en la Semana 12. El vehículo fue spawnado correctamente dentro del pasillo en el origen del mundo, y los topics `/scan`, `/cmd_vel` y `/odom` operaron según lo esperado. Se realizaron pruebas de teleoperación con el paquete `teleop_twist_keyboard` para validar el rango de maniobra del vehículo dentro del ancho disponible del pasillo, confirmando la viabilidad de la cinemática Ackermann en este entorno.



[Captura: DeepRacer spawnado dentro del pasillo USTA con el LiDAR activo, mostrando los rayos azules del sensor sobre las paredes]

Figura 2: Vehículo AWS DeepRacer spawnado dentro del gemelo digital del pasillo USTA. Se observan las paredes modeladas y los rayos del LiDAR sobre las superficies del entorno.

Este resultado cierra formalmente las actividades pendientes de Fase 3 mencionadas en el cronograma (identificación de módulos, representaciones conceptuales y definición conceptual

del funcionamiento del entorno virtual).

4. Implementación del wall follower

4.1. Contexto y justificación

A partir de una sugerencia de la dirección del proyecto, se planteó como primer ejercicio de Fase 4 la implementación de un comportamiento reactivo de seguimiento de pared (*wall following*) sobre el DeepRacer. Si bien este comportamiento no forma parte de los componentes del sistema final descritos en el anteproyecto (el cual se basa en navegación por grafo topológico sobre un mapa SLAM), constituye un ejercicio de integración entre los subsistemas de percepción (LiDAR) y control del DeepRacer, y permite validar prácticamente la respuesta del plugin Ackermann en condiciones de operación autónoma.

El enfoque de trabajo consistió en **aplicar algoritmos existentes en la literatura** sobre la plataforma del proyecto, evaluando empíricamente su comportamiento en el entorno simulado. No se trata de la derivación o diseño de algoritmos nuevos, sino de la implementación e integración de técnicas ya validadas por la comunidad académica.

4.2. Creación del paquete ROS 2

Se creó el paquete `wall_follower` bajo el workspace `deepracer_sim_ws`, utilizando el tipo de build `ament_python` y declarando como dependencias `rclpy`, `sensor_msgs` y `geometry_msgs`.

```
ros2 pkg create --build-type ament_python --license Apache-2.0 \
  wall_follower --dependencies rclpy sensor_msgs geometry_msgs
```

Listing 1: Comando de creación del paquete ROS 2.

La estructura interna del paquete sigue las convenciones estándar de ROS 2: un nodo Python (`wall_follower_node.py`) registrado como *entry point* en `setup.py`, lo cual permite su ejecución mediante `ros2 run wall_follower wall_follower_node`.

4.3. Primer enfoque: control proporcional-derivativo con un rayo del LiDAR

La primera implementación corresponde al enfoque clásico de wall following basado en un controlador proporcional-derivativo (PD) que utiliza un único rayo del LiDAR (correspondiente al lateral derecho del robot, a $-\pi/2$ rad en el frame del sensor). El nodo toma como entrada la distancia medida por ese rayo, calcula el error respecto a una distancia objetivo de 0.5 m, y publica una velocidad angular como salida del controlador hacia el topic `/cmd_vel`.

Durante la sintonización experimental se evaluaron diversas combinaciones de ganancias y velocidades lineales. El enfoque presentó dos limitaciones identificadas durante las pruebas: saturación frecuente de la velocidad angular en presencia de errores grandes, y ausencia de información sobre la orientación del robot respecto a la pared, lo cual dificulta mantener trayectorias estables a lo largo del recorrido.

4.4. Segundo enfoque: aplicación de la técnica F1TENTH

Como respuesta a las limitaciones identificadas se adoptó la técnica de wall following documentada en el curso académico F1TENTH [1, 2], ampliamente utilizada en vehículos Ackermann a escala y referenciada en diversos trabajos académicos sobre vehículos autónomos. La técnica fue **implementada tal como está documentada en el material del curso**, sin modificaciones al algoritmo.

A diferencia del primer enfoque, F1TENTH utiliza **dos rayos del LiDAR** (a $-\pi/2$ y $-\pi/4$ rad) en lugar de uno solo. Con estos dos rayos el algoritmo estima dos parámetros adicionales: el ángulo entre la orientación del robot y la pared, y una predicción de la distancia futura del robot a la pared tras avanzar una distancia configurable de *lookahead*. Esta información adicional permite que el controlador anticipe correcciones, lo cual produce un comportamiento más suave que el enfoque de un solo rayo.

Los parámetros del algoritmo configurados fueron: distancia objetivo a la pared de 0.5 m, distancia de lookahead de 1.0 m y velocidad lineal constante de 0.5 m/s. Las ganancias del controlador PD se ajustaron experimentalmente.

4.5. Validación experimental y resultados obtenidos

El comportamiento se evaluó en dos configuraciones de entorno virtual, con resultados sustancialmente distintos.

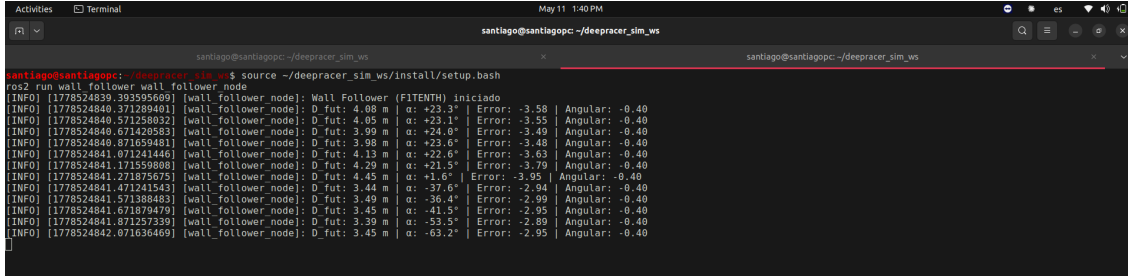
Pasillo original (7 m × 3 m): En esta configuración el wall follower **no logró un comportamiento estable**. Durante las pruebas se observaron tres patrones recurrentes:

- Saturación constante de la velocidad angular del controlador, lo que producía giros excesivamente cerrados respecto al radio de giro mínimo realizable por la cinemática Ackermann del vehículo.
- Trayectorias en círculos sin avance efectivo sobre el largo del pasillo.
- Colisiones del vehículo contra las paredes laterales antes de completar un recorrido significativo.

Se realizaron múltiples iteraciones de sintonización modificando las ganancias del controlador PD, la velocidad lineal de avance y los límites de saturación angular, sin obtener un comportamiento estable en este entorno. La hipótesis técnica que explica este resultado es que la relación entre el radio de giro mínimo del DeepRacer simulado y el ancho del pasillo (3 m) deja muy poco margen físico para que el controlador realice correcciones sin alcanzar la saturación.

Pasillo ampliado (15 m × 5 m): Para evaluar el algoritmo en un escenario con mayor margen físico se generó adicionalmente el modelo `pasillo_grande`. En esta configuración el algoritmo F1TENTH mostró **convergencia parcial** hacia la distancia objetivo: durante un segmento del recorrido la distancia futura predicha se estabilizó alrededor de 0.76 m (cercana al objetivo de 0.5 m, con un error sostenido de aproximadamente -0.26 m) durante varios segundos consecutivos. La Figura 3 presenta evidencia experimental de este régimen.

Sin embargo, esta convergencia no fue sostenida durante todo el recorrido. Al aproximarse a los extremos del pasillo, el vehículo perdía la referencia de la pared lateral (debido a la cobertura angular limitada del LiDAR del DeepRacer, de 300°), y el comportamiento divergía hasta detenerse contra alguna pared o salirse del entorno modelado. La fase transitoria inicial también resultó sensible a la posición y orientación de spawn del vehículo: posiciones iniciales mal alineadas con la pared generaban comportamientos no recuperables durante los primeros segundos.



```
santiago@santiagopc:~/deep racer_sim_ws$ source ~/install/setup.bash
ros2 run wall_follower wall_follower_node
[INFO] [1778524839.393595609] [wall_follower_node]: Wall Follower (F1TENTH) iniciado
[INFO] [1778524840.371289401] [wall_follower_node]: D fut: 4.08 m | α: +23.3° | Error: -3.58 | Angular: -0.40
[INFO] [1778524840.571288032] [wall_follower_node]: D fut: 4.05 m | α: +23.1° | Error: -3.55 | Angular: -0.40
[INFO] [1778524840.671288583] [wall_follower_node]: D fut: 3.99 m | α: +24.0° | Error: -3.49 | Angular: -0.40
[INFO] [1778524840.871259481] [wall_follower_node]: D fut: 3.98 m | α: +23.6° | Error: -3.48 | Angular: -0.40
[INFO] [1778524841.071241446] [wall_follower_node]: D fut: 4.13 m | α: +22.6° | Error: -3.63 | Angular: -0.40
[INFO] [1778524841.171559808] [wall_follower_node]: D fut: 4.29 m | α: +21.5° | Error: -3.79 | Angular: -0.40
[INFO] [1778524841.271275675] [wall_follower_node]: D fut: 4.45 m | α: +1.6° | Error: -3.95 | Angular: -0.40
[INFO] [1778524841.471241543] [wall_follower_node]: D fut: 3.44 m | α: -37.6° | Error: -2.94 | Angular: -0.40
[INFO] [1778524841.571388483] [wall_follower_node]: D fut: 3.49 m | α: -36.4° | Error: -2.99 | Angular: -0.40
[INFO] [1778524841.671279472] [wall_follower_node]: D fut: 3.45 m | α: -41.5° | Error: -2.95 | Angular: -0.40
[INFO] [1778524841.871257339] [wall_follower_node]: D fut: 3.39 m | α: -53.5° | Error: -2.89 | Angular: -0.40
[INFO] [1778524842.071636469] [wall_follower_node]: D fut: 3.45 m | α: -63.2° | Error: -2.95 | Angular: -0.40
```

[Captura de la terminal con los logs del wall follower F1TENTH durante la convergencia: la distancia futura decreciendo desde valores cercanos a 1.4 m hasta estabilizarse en torno a 0.76 m, con error cercano a -0.26 m]

Figura 3: Salida del nodo `wall_follower_node` durante el régimen de convergencia en el pasillo ampliado.

5. Evaluación del repositorio `deep racer-custom-car`

5.1. Origen y contenido del repositorio

A partir de una sugerencia de la codirección del proyecto, se evaluó el repositorio `aws-deepracer-community` [3], mantenido por la comunidad alrededor del AWS DeepRacer. El análisis del repositorio reveló las siguientes características relevantes para el proyecto:

- El repositorio está orientado a la ejecución del stack del DeepRacer sobre **hardware no original**, en particular Raspberry Pi 4 sobre Ubuntu 22.04 con ROS 2 Humble, así como Raspberry Pi 5 sobre Ubuntu 24.04 con ROS 2 Jazzy.
- **No contiene componentes de simulación:** no incluye URDF para Gazebo, plugins de simulación ni archivos de mundo. En consecuencia no reemplaza al repositorio actualmente utilizado para simulación.
- Incluye soporte nativo para LiDAR (RPLIDAR) y cámara, así como scripts de instalación automatizada para la Raspberry Pi 4.
- Cuenta con planos y documentación para construir un vehículo compatible con el DeepRacer a partir de un chasis WLToys y componentes accesibles, lo cual amplía las opciones de implementación física para el proyecto.

5.2. Distribución de repositorios en el proyecto

A partir del análisis se concluye que ambos repositorios son **complementarios** dentro del alcance del proyecto y resuelven necesidades distintas según la fase de desarrollo. La Tabla 1 resume la distribución de uso prevista.

Tabla 1: Distribución de los repositorios según la fase del proyecto.

Repositorio	Función	Fase del proyecto
<code>aws-deepracer/aws-deepracer</code>	Simulación en Gazebo Classic 11	Fases 3–4 (actual)
<code>aws-deepracer-community/deepracer-custom-car</code>	Ejecución sobre Raspberry Pi 4	Fase 5–6 (futura)

Esta decisión mantiene la coherencia con el principio de bajo costo y código abierto del anteproyecto, y se alinea con la estrategia de reemplazo de la unidad de cómputo original del DeepRacer por una Raspberry Pi 4 que fue formalizada en el informe de la Semana 12.

6. Lecciones aprendidas y limitaciones del ejercicio

El proceso de implementación y validación del wall follower permitió identificar limitaciones técnicas relevantes para el proyecto, así como aprendizajes que se documentan a continuación:

- **Inadecuación de los comportamientos puramente reactivos para Ackermann en espacios estrechos:** El comportamiento reactivo, al depender únicamente del estado instantáneo del sensor, no tiene capacidad de anticipar las restricciones cinemáticas del vehículo Ackermann. En espacios donde el radio de giro mínimo es comparable al ancho disponible, el controlador no logra realizar correcciones sin saturar. Esta limitación es estructural del enfoque y no se resuelve con sintonización adicional del PD.
- **Necesidad de mayor margen físico:** La validación parcial obtenida en el pasillo ampliado ($15\text{ m} \times 5\text{ m}$) sugiere que el algoritmo F1TENTH requiere un entorno con al menos varias veces el radio de giro del vehículo para operar con cierta estabilidad. Esto debe considerarse al diseñar los entornos de pruebas finales del proyecto.
- **Cobertura angular del LiDAR:** El sensor LiDAR del DeepRacer tiene una cobertura de 300° (no 360°), con una zona ciega de 60° en la parte trasera del vehículo. Esta limitación es relevante para cualquier comportamiento que requiera percepción completa del entorno, incluyendo SLAM y navegación con obstáculos dinámicos.

- **Saturación del plugin de simulación:** El plugin `deepracer_drive_plugin` presenta comportamientos de saturación en el ángulo de dirección durante transitorios con errores grandes. Si bien este fenómeno también ocurriría en hardware real, su manifestación específica en simulación debe tenerse presente al interpretar los resultados experimentales.
- **Sintonización iterativa:** La sintonización del controlador PD resultó significativamente más sensible de lo previsto. Múltiples combinaciones de ganancias y velocidades no produjeron comportamiento estable en el pasillo original, lo que evidencia que el problema no era únicamente de ajuste paramétrico sino de adecuación del enfoque al entorno.

Es importante destacar que estas limitaciones **no afectan negativamente al sistema final descrito en el anteproyecto**. El sistema final no se basa en comportamientos reactivos puros sino en navegación planificada sobre un mapa, como se detalla en la siguiente sección.

7. Reflexión sobre el movimiento autónomo del DeepRacer

El resultado parcial del wall follower abre una pregunta legítima sobre cuál será el enfoque de movimiento autónomo del DeepRacer en las semanas siguientes. Resulta importante aclarar que el proyecto se encuentra todavía en una fase exploratoria respecto a este componente, y que aún no se ha tomado una decisión definitiva sobre la arquitectura final de navegación.

El anteproyecto plantea de forma general una navegación basada en planificación sobre un mapa del entorno, distinta a los comportamientos puramente reactivos como el wall follower. En esa dirección, las semanas siguientes contemplan la exploración de varias herramientas y técnicas que serán evaluadas progresivamente:

- **Generación de un mapa 2D del entorno** mediante técnicas de SLAM. Se evaluará inicialmente la librería `slam_toolbox` por ser la opción de referencia en ROS 2, aunque su comportamiento sobre el DeepRacer y el LiDAR de cobertura 300° aún debe verificarse experimentalmente.

- **Localización del vehículo sobre el mapa** una vez generado. AMCL (Adaptive Monte Carlo Localization) es la opción más documentada para esta tarea, pero su desempeño sobre el DeepRacer en simulación deberá probarse antes de adoptarse formalmente.
- **Definición de un grafo topológico de puntos de interés** sobre el mapa, mencionado en el anteproyecto como base para la planificación de rutas. El esquema concreto de representación del grafo está aún por definir.
- **Planificación de rutas con cinemática Ackermann.** En el informe de la Semana 11 se mencionaron como candidatos los planificadores locales **TEB Local Planner** y **Smac Hybrid-A***, ambos compatibles con vehículos tipo automóvil. La selección entre estos u otros planificadores se realizará a partir de pruebas comparativas en simulación.

Por tanto, las próximas semanas tendrán un carácter exploratorio: se irán evaluando estas herramientas de forma incremental y los resultados experimentales determinarán cuáles se adoptan en el sistema final. Lo que el ejercicio del wall follower ha permitido establecer con claridad es que los comportamientos puramente reactivos presentan limitaciones importantes para vehículos Ackermann en espacios estrechos, lo cual orienta la exploración hacia enfoques de navegación con información global del entorno, sin que ello implique aún un compromiso definitivo con una arquitectura específica.

8. Estado del cronograma y trabajo pendiente

8.1. Estado actual de actividades

El estado de las actividades correspondientes al inicio de Fase 4 se resume en la Tabla 2. Las actividades de Fase 3 fueron cerradas en la Semana 12; las actividades de Fase 4 listadas a continuación corresponden al rango S13–S17 según el cronograma del anteproyecto.

Tabla 2: Estado de actividades — Inicio de Fase 4.

Actividad	Estado	Observación
Modelado del entorno virtual	Completado	Pasillo USTA a escala 7×3 m.
Validación del DeepRacer en entorno virtual	Completado	Spawn y teleoperación verificados.
Implementación de wall follower	Parcial	Convergencia parcial en pasillo ampliado.
Evaluación de <code>deepracer-custom-car</code>	Completado	Decisión documentada.

8.2. Trabajo previsto para semanas siguientes

De acuerdo con el cronograma del anteproyecto, las actividades correspondientes a la **Fase 4 (Desarrollo progresivo de la solución, S13–S17)** previstas para las semanas siguientes son las siguientes. Estas actividades se abordarán con carácter exploratorio: los resultados experimentales obtenidos en cada paso orientarán las decisiones técnicas sobre los componentes del sistema final.

- **S14:** Exploración inicial de técnicas de SLAM sobre el entorno virtual, evaluando la generación de un mapa 2D del pasillo a partir de las lecturas del LiDAR del DeepRacer.
- **S14–S15:** Exploración de esquemas para representar puntos de interés sobre el mapa generado, candidatos para soportar la planificación de rutas del sistema.
- **S14–S15:** Pruebas de instanciación simultánea de los dos agentes en simulación mediante *namespaces* ROS 2.
- **S15–S16:** Diseño y pruebas iniciales del servidor central de coordinación y del protocolo de relevo entre agentes.
- **S15–S17:** Desarrollo de la interfaz HRI web y de su mecanismo de integración con el servidor.

9. Conclusiones de la Semana

La Semana 13 entregó tres resultados verificables: el gemelo digital del pasillo de la Universidad Santo Tomás funcional en Gazebo Classic 11, una implementación del wall follower con la técnica F1TENTH evaluada experimentalmente, y la formalización del uso de dos repositorios complementarios para las fases de simulación y de hardware real.

El ejercicio del wall follower arrojó **resultados parciales**: el algoritmo F1TENTH no logró un comportamiento estable en el pasillo original (7 m $\times$ 3 m) debido a la combinación entre las restricciones cinemáticas del DeepRacer y el ancho disponible del entorno. En un pasillo ampliado (15 m $\times$ 5 m) generado específicamente para evaluar el algoritmo, se observó convergencia parcial hacia la distancia objetivo durante segmentos del recorrido, sin que el comportamiento se mantuviera estable durante el recorrido completo. Este resultado, aunque

no produce un demo de wall following plenamente funcional, aporta evidencia técnica relevante respecto a las limitaciones de los comportamientos reactivos sobre vehículos Ackermann en espacios estrechos.

La principal lección de la semana es que la línea de exploración hacia el sistema final requiere apartarse del enfoque puramente reactivo y avanzar hacia enfoques que aprovechen información global del entorno. Para la Semana 14 se propone iniciar esta exploración mediante la generación de un mapa 2D del entorno virtual con técnicas de SLAM, evaluando experimentalmente su comportamiento sobre el DeepRacer simulado. Las decisiones sobre las herramientas específicas que conformarán la arquitectura final de navegación se irán tomando progresivamente a partir de los resultados experimentales obtenidos en las semanas siguientes.

Referencias

- [1] M. O’Kelly, V. Sukhil, H. Abbas, et al., “F1/10: An Open-Source Autonomous Cyber-Physical Platform,” arXiv:1901.08567, 2019. [Online]. Available: <https://arxiv.org/abs/1901.08567>
- [2] University of Pennsylvania, “F1TENTH Autonomous Racing Course — Lab 3: Wall Following,” [Online]. Available: <https://f1tenth.org/learn.html>
- [3] AWS DeepRacer Community, “deepracer-custom-car: Custom DeepRacer build with Raspberry Pi support,” GitHub repository, 2024–2026. [Online]. Available: <https://github.com/aws-deepracer-community/deepracer-custom-car>
- [4] Open Robotics, “ROS 2 Humble Hawksbill Documentation,” [Online]. Available: <https://docs.ros.org/en/humble/>